

DEVELOPING A LIVE 3D TELEMTRY VISUALIZATION FOR STAIRLIFT FAULT DIAGNOSIS

Nout Mulder

DeVi Comfort • Engineering • Opmeer

Abstract. This research addresses the challenge of transforming raw stairlift telemetry into a stable, accurate live 3D visualization for fault diagnosis. The core technical problem is the mapping of digital pulse-encoder readings, polled over a high-latency Bluetooth link, onto a curved 3D rail geometry with segment-dependent encoder-to-distance ratios, while keeping the rendered motion smooth on the display loop. A “virtual ghost” prototype was developed iteratively and validated on physical stairlift hardware across two rail configurations (3403 mm and 5428 mm) against pre-registered quantitative criteria. Position accuracy was evaluated at two levels: the pipeline-integration error (display vs. firmware-encoded position from the same encoder reading) stayed within 0.7 mm on both rails, while the end-to-end display-vs-physical error on the 5428 mm rail reached at most 9.9 mm (inside the 10 mm tolerance for rails longer than 5 m), with the gap between the two measures most plausibly attributable to the upstream rail-configurator rather than to the pipeline. Bluetooth round-trip latency met the pre-registered percentile thresholds on both rails ($P95 \leq 113$ ms, $P99 \leq 135$ ms; thresholds 150/250 ms), and a dedicated continuous-drive log showed no rendering staircase (velocity spikes on 0.04 % of frames on the steady bottom-to-top leg, more than two orders of magnitude below the last-value-hold signature the criterion is designed to detect). The results show that a processing pipeline combining non-linear encoder mapping, recursive state estimation with dead-reckoning, and smooth tangent-based orientation reconstruction produces a stable, accurate live 3D representation of stairlift position, with fault state rendered directly from the parsed status bits.

1 Introduction

DeVi Comfort designs and manufactures stairlift systems and develops the accompanying control software in-house. A stairlift consists of a chair unit that travels along a custom-bent rail mounted to a staircase. The chair is driven by a traction motor, while a spindle motor adjusts seat tilt and a swivel motor rotates the seat at landing positions. During operation, embedded controllers continuously produce telemetry: traction encoder pulses, spindle encoder values, swivel angles, and system status bits including error codes. For service, maintenance, and further development, engineers need to quickly understand the current state of a stairlift: where the chair is positioned on the rail, whether it is moving correctly, and what deviations or faults have occurred.

Currently, fault information from the controller is presented as textual error codes and numeric status values. While technically correct, these require significant domain knowledge to interpret spatially. An error code indicating a position deviation does not convey where on the rail the deviation occurred, in which direction, or how large the discrepancy is.

The underlying technical challenge is twofold. The raw telemetry data (encoder pulse counts, spindle values, swivel angles) must be mapped onto a three-dimensional rail geometry. This rail contains straight sections, vertical bends, and horizontal bends, each consuming a different number of encoder pulses per millimeter of travel. Additionally, the sensor readings arrive at irregular intervals over a wireless serial link with noise, latency, and occasional packet loss. The visualization must bridge the gap between these asynchronous measurements and a contin-

uous, smooth spatial representation that updates at display frame rate.

A live 3D visualization of stairlift position and movement addresses this gap directly: it makes the chair’s spatial state (position on the rail, motion through bends, and seat tilt) explicit where the current text-and-code output leaves it implicit. This gives service engineers an at-a-glance view of where the chair is and how it is oriented, and gives the development team a direct way to verify firmware behaviour during testing and commissioning of new rail configurations. Whether this spatial view in turn shortens diagnosis time or reduces on-site visits is the functional motivation for the work, examined as a human-factors question outside the present scope.

The goal of this research is to determine which data processing and mathematical models are needed to transform live stairlift telemetry into a stable and accurate 3D visualization. Whether such a visualization ultimately accelerates fault diagnosis compared to text-based error codes is a practical motivation for the work, but lies outside the scope of this report; the central research question is therefore restricted to what can be deductively established:

What data processing pipeline and mathematical models are required to transform live stairlift telemetry into a stable, accurate 3D visualization?

The research follows an iterative prototyping approach based on the spiral model [1]. Each iteration consists of analysis, design, implementation, and validation against practical scenarios on physical hardware. The primary methods are formal mathematical analysis, architectural design, and experiment with measurable accuracy criteria.

Section 2 presents the problem statement, structured as a problem analysis, prior work survey, and the resulting research questions. Section 3 describes the research methods per subquestion. Section 4 presents the findings. Section 5 concludes with answers to the research questions and recommendations.

2 Problem Statement

2.1 Problem Analysis

2.1.1 Sensor-to-Spatial Mapping in Mechatronic Systems

The challenge of transforming discrete sensor readings into a continuous spatial representation is well-established in mechatronic system diagnostics [2]. In the broader context of digital twin research, Bofill et al. identify real-time sensor data integration as one of the primary technical barriers to effective fault monitoring [3]. The fundamental difficulty is that sensors produce scalar values (counts, voltages, status bits) at discrete intervals, while a spatial visualization requires a continuous 3D position and orientation that updates at display frame rate.

In robotics and vehicle tracking, this problem is typically simplified by the assumption that sensor readings map linearly to spatial coordinates [4]. A rotary encoder on a linear actuator, for example, produces pulses proportional to displacement, and the mapping from pulse count to position is a single constant. This assumption breaks down when the trajectory is non-linear: on a curved path, the relationship between encoder pulses and spatial displacement depends on the local curvature [5].

2.1.2 Non-Linear Encoder Mapping on Curved Rails

A stairlift rail is composed of straight sections, vertical bends (pitch changes), and horizontal bends (yaw changes). The mechanical drive system uses a traction motor with an incremental encoder to measure displacement [6]. Due to the geometry of the rail bending mechanism, each segment type consumes a different number of encoder pulses per unit of physical travel. A vertical upward bend, for instance, requires fewer pulses per millimeter of arc length than a downward bend, because the traction wheel follows the inner or outer radius of the curve respectively.

This means that a single scalar encoder reading cannot be converted to a 3D position without a model of the full rail topology. The mapping must account for the cumulative pulse cost of each preceding segment, making it a piecewise non-linear function rather than a simple linear scaling. Without such a model, position errors accumulate at every bend transition, growing proportionally to the number of bends in the rail.

2.1.3 State Estimation from Asynchronous Telemetry

The telemetry link between the stairlift controller and the visualization application introduces additional challenges. The Bluetooth Classic Serial Port Profile [7] pro-

vides a virtual serial connection at approximately 20 Hz; the round-trip time at the application layer depends on link-layer latency plus controller response time and was measured in the validation runs of Section 4.4. The visualization, however, must render at 60 fps (16.7 ms per frame).

This creates a temporal mismatch: new sensor data arrives many render frames apart, and frames between measurements must be filled with predicted values. If the most recent measurement is simply held constant until the next arrives, the display exhibits visible staircase-like jumps at the sensor sample rate. The traction encoder used by the stairlift is a digital incremental pulse-counter [5]: it does not produce per-sample quantization noise on a stationary chair, but it does produce sampled positions at a rate well below the display refresh, so the gap-filling problem (not noise rejection) is the dominant artifact source for this visualization.

This frames the technical requirement precisely: the visualization needs a continuous position-and-velocity estimate that can be advanced at display frame rate from samples arriving several times less often, and that can bridge the interval when a sample is late or missing. The required property is therefore gap-filling and velocity estimation between samples, not noise rejection on the individual sample. Which class of estimator satisfies this requirement, and at what cost, is a solution question addressed in the prior work survey (Section 2.2).

2.1.4 Multi-Axis Orientation from Independent Sensors

The stairlift chair has four independent rotational degrees of freedom: yaw (direction along the rail), pitch (rail inclination), seat tilt (spindle motor), and seat rotation (swivel motor). Each is measured by a separate sensor with its own update rate and noise characteristics. Reconstructing the full 3D orientation of the chair requires combining these independent streams into a single consistent rotation that stays well-defined for every reachable chair configuration, including those in which two of the rotation axes momentarily align. Whether a particular orientation representation can guarantee that property is a solution question, addressed in Section 2.2.

2.1.5 Summary of Technical Subproblems

Three interrelated technical subproblems emerge from this analysis:

1. **Non-linear encoder-to-position mapping:** Converting a scalar encoder count into a normalized position along a 3D curve with segment-dependent pulse costs.
2. **State estimation:** Producing a smooth, low-latency estimate of position and velocity from noisy, asynchronous measurements arriving at a lower rate than the display refresh.
3. **Multi-axis orientation reconstruction:** Combining four independent rotational sensor streams into a singularity-free 3D orientation.

2.2 Prior Work

2.2.1 Encoder-to-Position Mapping on Non-Uniform Paths

Converting an incremental encoder count to a position along the travel path is, for a uniform path, a solved problem: the encoder produces pulses proportional to displacement and a single scale constant suffices [6, 4]. Three solution directions exist for the non-uniform case identified in Section 2.1.

A **single linear scale factor** multiplies the pulse count by one constant mm-per-pulse value, the standard linear-actuator approach [4]. It is $O(1)$ per frame and needs no model of the path, but is correct only while the pulses-per-millimetre ratio is constant; [5] shows this assumption fails where the local path geometry varies, so on a bent rail the linear factor accumulates a position error at every bend transition. It is therefore rejected as the primary mapping.

A **closed-form arc-length parameterization** integrates the encoder-to-arc-length relation analytically when the path is a single analytic curve [8]. This is exact and compact, but a closed-form inverse exists only for a single uniform segment type (constant curvature). The stairlift rail mixes straight, vertical-bend and horizontal-bend segments with different per-segment pulse costs, so no single closed-form inverse covers the whole rail; the approach is usable only per segment, not rail-wide.

A **precomputed piecewise calibration map** tabulates, at each segment boundary, the cumulative encoder count against the cumulative Euclidean distance and interpolates within a segment, reducing every per-frame query to a binary search over a static table [6]. It handles arbitrary per-segment pulse costs, amortises the path integration to once per rail, and degrades gracefully (piecewise-linear) between boundaries. Its cost is that the full rail topology must be known up front, which it is, since the controller stores the rail as typed blocks. This direction is selected and derived in Section 4.2.

2.2.2 State Estimation

For state estimation from noisy sensor data, three approaches are commonly used in embedded and robotics applications: moving average filters, complementary filters, and Kalman filters.

A **moving average filter** computes the mean of the last N samples and outputs it as the filtered value. Smith shows that for purely time-domain noise reduction on stationary or slowly varying signals, moving averages are close to optimal and have the advantage of computational simplicity [9]. They are therefore a strong choice for applications where the input signal changes slowly relative to the sampling rate and no prediction is required between samples. However, moving averages introduce a latency proportional to the window size (effectively delaying the signal by $N/2$ samples), cannot extrapolate between measurements, and perform poorly on transient or fast-changing signals. For stairlift telemetry, where per-channel sensor readings arrive several times less often than the 60 fps display refresh, pure moving-average smoothing cannot fill the inter-sample gap.

A **complementary filter** combines two sensor sources with complementary frequency characteristics, typically using a low-pass filter on one source and a high-pass filter on the other [10]. This is the standard approach in IMU fusion, where slow, drift-free accelerometer data is combined with fast, noisy gyroscope data to estimate attitude. Complementary filters are computationally light and do not require a statistical model of the noise. They are therefore preferred in cost-constrained or high-rate embedded systems with multiple complementary sensors. However, they assume that two sensor sources are available with complementary frequency content, which is not the case for the stairlift: the traction encoder is the only direct position sensor, and its noise is not separable by frequency from its signal.

The **Kalman filter** maintains a probabilistic estimate of system state (position and velocity) and updates it with each new measurement, optimally balancing prediction uncertainty against measurement noise [11, 12, 6]. It combines state estimation and prediction in a single framework, which makes it a natural fit when measurements arrive at a lower rate than the required output. The cost is higher computational complexity and the need to tune process and measurement noise parameters. For the stairlift application, the Kalman filter is selected because it is the only approach of the three that can both estimate a continuous velocity from the discrete pulse-counter stream and predict position between measurements, both of which are required to feed a 60 fps display from a per-channel sensor stream that arrives several times more slowly than the display refresh, without visible staircase-like jumps at the sample rate.

2.2.3 Spline Interpolation for Smooth Trajectories

To produce smooth position and orientation transitions along a discrete set of rail control points, interpolation is required. Three families of splines are candidates.

Linear interpolation between adjacent points produces sharp direction changes at segment boundaries (no C^1 continuity). It is computationally cheap and is the right choice when the interpolated quantity is expected to change piecewise linearly, or when downstream processing does not need a continuous tangent. For visual orientation of a chair along a continuously curved rail, the discontinuous tangent of linear interpolation would cause the chair to visibly “snap” at each rail point.

B-splines (and their rational generalization, NURBS) are the de facto standard in CAD and industrial curve design. They provide arbitrary continuity (C^k) through the choice of degree, and local control (moving one control point affects only a bounded region of the curve) [13]. They are preferred when the curve itself is being designed (CAD surfaces, font glyphs, animation paths) and the control points are construction handles rather than fixed waypoints. Their disadvantage for this application is that the curve does *not* pass through the control points, so it does not correspond to the physical rail geometry: the ghost chair would follow a smoothed approximation of the rail rather than the rail itself.

Catmull-Rom splines interpolate through each control point and provide C^1 continuity at those points [14]. They

are the natural choice when the control points are fixed physical waypoints (as is the case for the rail) and the primary goal is a smooth tangent for orientation purposes. They are selected for this work specifically as the *tangent* source for chair orientation; the chair *position* along each segment is computed by linear interpolation between adjacent rail control points, which preserves the on-rail constraint trivially because both linear and Catmull-Rom curves pass through the control points. The combined scheme (linear position, Catmull-Rom tangent) gives smooth chair orientation without introducing the off-rail excursions that a full Catmull-Rom positional curve would produce between sparsely sampled points.

2.2.4 Rotation Representation

Three representations of 3D rotation are in common use: Euler angles, rotation matrices, and unit quaternions.

Euler angles (roll, pitch, yaw) are human-readable and require only three parameters. Diebel notes that for systems with limited rotation ranges, or where only a single rotation axis is actively used, Euler angles are a reasonable and computationally inexpensive choice [15]. Many industrial applications (e.g., single-axis servo control, simple tilt sensors) use Euler angles without issue. The well-known drawback is gimbal lock: when two rotation axes align, one degree of freedom is lost and the inverse mapping becomes singular. For the stairlift, with four independent rotational axes that can interact (rail yaw, rail pitch, seat tilt, seat rotation), the risk of gimbal lock during certain rail configurations is real.

Rotation matrices (3×3 orthogonal matrices) are free of singularities and compose cleanly, but require nine parameters with six constraints, are costly to interpolate smoothly, and accumulate numerical drift (loss of orthogonality) over many composed rotations. They are standard in computer graphics shaders, where the GPU performs matrix operations natively.

Unit quaternions (four parameters, one constraint) are free of gimbal lock, compose compactly, and support smooth spherical linear interpolation [16]. They are the standard in real-time graphics and robotics for full 3D orientation [17, 15]. For the stairlift chair, where four independent rotations must be composed every frame without visual artifacts, quaternions are selected to avoid the gimbal lock risk of Euler angles and the numerical drift of rotation matrices.

2.2.5 Telemetry Transport Link

The input layer of the pipeline (Section 4.1) must carry telemetry from a controller mounted on the moving chair to an off-board visualization host, which rules out a tethered link from the outset: a wired serial (UART/RS-232) or Ethernet connection would have to be dragged along the travelling carriage. A wireless link is therefore a hard requirement. The transport is, however, not a free design variable in this project but a property inherited from the deployed hardware: the lift controller carries an HC-series Bluetooth-to-serial module and has historically communicated with companion applications over that same Bluetooth-serial link. The pipeline conse-

quently takes Bluetooth Classic with the Serial Port Profile [7] as a given input rather than selecting it against alternatives such as Wi-Fi, which would require an IP stack and network association the controller does not provide. A TCP transport is implemented behind the same interface, but only as a development and test-bench substitute when no physical lift is connected. The round-trip latency this inherited link imposes is precisely what makes the inter-sample prediction of Section 4.3 load-bearing, and characterising it is part of the first subquestion (Section 2.3).

2.2.6 Real-Time 3D Engine Selection

The renderer layer of the pipeline (Section 4.1) imposes four *mandatory* deployment constraints on the 3D engine, each a hard requirement rather than a weighted preference: (C1) configurability for custom shaders, (C2) cross-platform export to desktop and Android 12+ from a single source tree, (C3) a licence compatible with closed-source commercial deployment, and (C4) native glTF binary import to match DeVi Comfort’s CAD pipeline [18]. Because these are constraints to be satisfied rather than trade-offs to be balanced, the selection is a feasibility screen (does an engine meet every constraint?) rather than a weighted multi-criteria trade-off; a partial match on a mandatory constraint counts as a failure. Table 1 screens four candidates against them.

Table 1: Feasibility screen of real-time 3D engines against the four mandatory deployment constraints defined in Section 2.2.6. “+” indicates the constraint is fully met, “=” partially, “-” not met; only an engine meeting all four is feasible.

Engine	C1	C2	C3	C4
Godot 4.6 [19]	+	+	+	+
Unity 6 [20]	=	+	=	+
Unreal Engine 5 [21]	+	=	=	+
Three.js + WebGL [22]	=	-	+	+

Godot 4.6 is selected: it is the only candidate satisfying all four mandatory constraints. Its MIT licence [19] avoids royalty and seat-fee risk; its export pipeline compiles a single source tree to Windows, Linux, and Android. Unreal was excluded on licence (5% royalty above a per-product annual revenue threshold [21]) and mobile export friction; Unity on its seat-fee and revenue-threshold model [20] combined with licence-stability risk after the 2023 runtime-fee announcement [23]; Three.js on lack of a native Android build target, which would force a WebView wrapper and break the gyroscope/accelerometer access required for the compass tracker (Section 4.4).

2.2.7 Existing Work at DeVi Comfort

No spatial visualization of stairlift telemetry existed at DeVi Comfort prior to this research; fault output is described in Section 1. The IPC (Inter-Process Communication) infrastructure connecting the configurator application to the lift controller was limited to rail upload and download, so the input layer of the proposed pipeline

(live telemetry polling) had no existing implementation to build on.

2.3 Research Questions

The problem analysis identified three technical subproblems (non-linear encoder mapping, state estimation, and multi-axis orientation reconstruction), while the prior work survey identified candidate approaches for each. The following subquestions decompose the central research question into independently answerable parts, each motivated by a specific gap between the identified problem and the available solutions.

1. *What data pipeline architecture transports live telemetry from the controller to a 3D rendering environment within real-time constraints?* This question subsumes the identification of which firmware messages (out of over 140 defined types) carry spatially interpretable information, because the input layer of the pipeline is precisely that subset. The Bluetooth serial link introduces latency, noise, and packet loss, which the pipeline must handle while keeping pace with the sensors as closely as the shared link allows.
2. *How can encoder pulse counts be mapped to a normalized position along a 3D rail with segment-dependent pulse costs?* As established in Section 2.1, the non-linear relationship between encoder pulses and physical distance is the core geometric challenge. No existing mapping exists for this rail type.
3. *Which state estimation and orientation models produce a stable live 3D representation from telemetry sampled asynchronously and well below the display refresh rate?* The prior work survey identified several candidate models per subproblem. This question addresses the formal derivation, parameterization, and empirical evaluation of the selected models for this specific application.

The empirical evaluation of each subquestion against measurable thresholds is treated as a validation methodology that applies across all three, rather than as a separate research question. It is described in Section 3.4.

3 Methods

Each subquestion is addressed with one or more research methods. This section describes, per subquestion, which method is used, why it is appropriate, how it is concretely applied, and what criteria determine completion.

3.1 Data Pipeline Architecture (Design + Documentation Analysis)

Method: Architectural design, combined with formal analysis of the firmware protocol specification to define the input layer.

Why this method: The telemetry pipeline spans multiple layers (wireless transport, binary protocol parsing, polling, state estimation, rendering) and must poll the position-relevant channels as fast as the shared Bluetooth link allows while keeping the round-trip time within a budget compatible with a 60 fps render loop. A structured architectural design ensures that each layer's contribution

to latency and reliability is identifiable. The input to that pipeline is a subset of the firmware's > 140 message types, which is determined by documentation analysis rather than observation of live traffic: the input is the complete firmware enumeration, not sampled message frames.

Approach: The architecture is specified as a layered pipeline (transport, protocol parser, telemetry poller, state estimator, 3D renderer). Each layer is documented with its inputs, outputs, and failure modes. As input to the design, the message enumeration in the controller firmware source is extracted and each message type is classified along three axes: (1) data content, (2) update rate, and (3) spatial interpretability.

Why this approach: A five-layer decomposition is chosen (rather than a flat producer/consumer or a two-tier client/renderer) because each identified failure mode (bit-level corruption, message-level corruption, connection drops, sensor noise, frame drops) maps cleanly onto exactly one layer. A flatter split would force the renderer to handle protocol errors or the transport to know about state estimation, collapsing concerns that the timing budget already separates. The three-axis message classification is chosen over a single-axis "relevant yes/no" because content selects parsing logic, rate selects polling strategy, and spatial interpretability selects relevance to the visualization.

Instrument (when done): The design is considered sufficient when: (a) every message ID in the firmware enumeration has been classified, with the subset relevant to the visualization pipeline explicitly identified with rationale and documented at byte-level for any message with a position, angle, or status bit field; and (b) the Bluetooth round-trip latency stays within the pre-registered percentile bounds ($P95 < 150$ ms and $P99 < 250$ ms) over each run, with the chair on screen additionally observed to track the physical chair during steady-state motion without perceptible lag (DV1(b), Section 3.4). A percentile bound is used rather than a single-sample threshold because the pipeline explicitly masks per-sample latency at the display side (Section 4.3); the percentile form bounds the inter-sample gaps the dead-reckoning must bridge, while the visible-lag observation is reported alongside it as the end-user property the link criterion ultimately serves.

3.2 Encoder-to-Position Mapping (Formal Analysis)

Method: Formal mathematical analysis.

Why this method: The encoder-to-position mapping is a deterministic geometric problem: given a rail composed of segments with known curvature and known pulse costs per segment type, the mapping can be derived analytically. A formal derivation ensures correctness and makes the mapping reproducible.

Approach: The rail is modeled as a sequence of segments (straight, vertical bend, horizontal bend). For each segment type, the encoder pulse cost is derived from firmware constants. Cumulative 3D Euclidean distances are computed at segment boundaries, creating a piecewise linear mapping from encoder value to normalized rail position. The derivation includes the arc-tracing geometry used to construct 3D control points from the firmware's

segment description.

Why this approach: Storing a precomputed piecewise mapping at segment boundaries (instead of integrating the curve at every frame) is chosen because the mapping is static once the rail is known, while the encoder reading changes every frame; amortising the integration once per rail keeps the per-frame cost to a single binary search. The alternative of a single closed-form encoder-to-arc-length function would only exist for a single uniform segment type, which does not hold here.

Instrument (when done): The formal derivation is considered complete when: (a) the mapping is defined for all segment types in named equations traceable to firmware constants, and (b) the resulting displayed position deviates by less than 5 mm from the physically measured chair position at every reference point (station boundaries, each bend entry, each bend exit) on rails up to 5 m, and less than 10 mm on longer rails, across the validation runs of Section 3.4.

3.3 Mathematical Models (Formal Analysis)

Method: Formal mathematical analysis.

Why this method: The state estimation, interpolation, and rotation models are grounded in established mathematical theory (Kalman filtering, Catmull-Rom splines, quaternion algebra). Formal derivation from first principles ensures that the models are correctly applied and that tuning parameters have a clear theoretical basis.

Approach: Three models are derived:

- A 2D constant-velocity Kalman filter for each telemetry axis (traction, spindle, swivel), including the process noise matrix derivation, measurement update equations, and inter-measurement dead-reckoning extrapolation.
- Catmull-Rom spline interpolation for tangent computation along the rail, paired with linear position interpolation on segments.
- Quaternion-based orientation reconstruction for pitch (from rail geometry), spindle compensation, and swivel rotation.

Why this approach: The three models are derived independently, each with its own state and its own update equations, rather than as a single fused state vector. This is chosen because the three axes have distinct noise characteristics (traction encoder quantization, spindle lookup interpolation, swivel digitisation) and distinct time bases; a single fused filter would require a shared process noise model that averages away these differences. Per-axis constant-velocity filters allow tuning (σ_a, σ_m per axis, Table 4) to reflect each sensor's actual behaviour.

Instrument (when done): Each model is considered correct when (a) every step of the derivation is presented in named equations traceable to the cited sources, and (b) the rendered position passes the velocity-based DV3(b) smoothness test: during continuous motion the frame-to-frame velocity shows no spikes above $5\times$ the median $|v|$, the signature of a last-value-hold staircase (Section 3.4), so that the display fills the $\sim 4\text{--}5\text{ Hz}$ per-channel sample stream (bounded by the shared-link round-trip; Section 4.1) without visible step-jumps on the render loop. The Kalman filter in this pipeline functions primarily as a *temporal* interpolator and velocity estimator for the ren-

der loop, not as a noise-rejection filter: the traction encoder is a digital pulse-counter that produces no value-jitter on the sample itself, so the relevant quality is the smoothness of the filtered output as a continuous-time signal that fills the $\sim 200\text{ ms}$ gaps between sensor updates. Singularity-freeness of the quaternion composition is a theoretical property of the formalism [16, 17] and is therefore not separately measured; the absence of visible chair snapping during the validation runs is a sufficient observational check.

3.4 Validation Approach (Experiment)

The instrument criteria of Sections 3.1–3.3 are bundled under the labels DV1, DV2, and DV3 (*deelvalidatie* 1–3), one per research subquestion. Each is split into an analytical sub-criterion (a) (derivation traceable to literature or firmware constants) and an empirical sub-criterion (b) (measured on physical calibration runs). The empirical sub-criteria are listed in Table 2; the analytical sub-criteria are contained in the originating method sections.

The thresholds in Table 2 are the second-version (v2) criteria. The first measurement campaign (Section 4.4) exposed that the original v1 thresholds for DV1(b) and DV3(b) assumed behaviour this system does not exhibit: a single-sample 100 ms round-trip target that ignores the latency-masking effect of the state estimator, and a “Kalman jitter $\geq 10\times$ smaller than raw” ratio that is undefined for a digital pulse-encoder showing zero value-jitter at rest. The criteria were therefore reformulated into quantitative, sensor-appropriate forms (a percentile latency bound and a velocity-based smoothness test) and *pre-registered*, with a recorded version-control timestamp, before the second measurement campaign, so that the decisive runs on the longer rail are scored against thresholds fixed in advance. This refinement and its effect on evidentiary weight are stated explicitly under Limitations (Section 5). Beyond DV1–DV3, the orientation outputs (spindle seat-tilt and swivel angle) are checked in separate static tests against external references (a digital spirit level and the tablet gyroscope) and reported alongside the main criteria in Section 4.4.

Table 2: Empirical validation criteria DV1(b)–DV3(b) with their originating method section and pre-registered pass threshold. Sub-criterion (a) (analytical traceability) is stated in the originating method section.

ID	Origin	Threshold (pre-registered, v2)
DV1(b)	Section 3.1	Bluetooth round-trip P95 < 150 ms and P99 < 250 ms, per run
DV2(b)	Section 3.2	Display vs. physical position < 5 mm (rails $\leq 5\text{ m}$), < 10 mm (longer); secondary: display vs. firmware-encoded position (pipeline isolation)
DV3(b)	Section 3.3	0 velocity spikes $> 5\times$ the median $ v $ during continuous motion (no staircase)

Method: Experiment on physical stairlift installations

with predefined pass/fail thresholds.

Why this method: The mathematical models and pipeline architecture must be verified under real-world conditions where mechanical tolerances, Bluetooth variability, encoder quantization, and sensor noise all interact. Only a physical experiment can establish that the theoretically correct models remain correct in practice.

Approach: Calibration runs are executed on multiple stairlift installations with different rail configurations (varying length, number of bends, and bend types). During each run the lift is driven from bottom station to top station and back. The prototype logs raw encoder values, filtered positions, angle corrections at each bend, render frame rate, and Bluetooth round-trip time. Position error is measured by comparing the displayed position at known reference points (station boundaries, bend start/end) against the physical position of the chair using the measurement procedure described in Section 4.4.

Why this approach: Thresholds are defined before the runs, not after, and each is tied to one of the DV1–DV3 instrument criteria. Measuring at fixed reference points (station boundaries and bend entries/exits) rather than averaging over the full run is chosen because these are the points at which a mapping fault would be visible to an operator; an average over the full rail would dilute a localised error.

Instrument (when done): A run is reported with measurements against every DV1–DV3 criterion that requires empirical evidence. Pass/fail is assigned per criterion as defined in the originating method section. Runs that fail on any criterion are reported with the cause.

Observational notes recorded alongside the measurements:

- O1 (pipeline robustness): the pipeline sustains a continuous drive over the full rail in both directions without crashes, freezes, or UI corruption.
- O2 (visual quality through bends): the chair on screen tracks the rail through bends without visible snapping or jumps at segment boundaries.
- O3 (visible motion smoothness during steady-state driving): while the chair drives between stations at typical pilot speed, the rendered chair on screen tracks the physical chair without perceptible step-jumps or lag. This observation complements the quantitative DV1(b) and DV3(b) measures by confirming the end-user property they target (visible lag and smoothness), which a latency percentile and a frame-velocity statistic bound but do not themselves display.

4 Results

4.1 Data Pipeline Architecture

The input layer of the pipeline is the subset of the firmware’s message enumeration that carries spatially interpretable information. Analysis of the stairlift firmware protocol reveals 145 defined message types (with IDs ranging from 0 to 221), of which six are directly relevant for live spatial visualization. These are classified in Table 3.

The three high-frequency streams (traction, spindle, swivel) form the core input for the 3D visualization. They

are polled in an alternating pattern to share the single serial Bluetooth link: the fast loop is scheduled to tick at ~ 20 Hz and alternates between traction and spindle requests on successive ticks, while the swivel coroutine polls on the same link at 50 ms intervals. The achievable rate is bounded by the application-layer round-trip time, which was measured at mean 89 ms over the validation runs (Section 4.4): because each request and its response must complete before the next request is issued on the shared link, the *aggregate* round-trip throughput is ~ 10 Hz (5295 round-trips over the 524 s of driving), not the nominal 20 Hz. That aggregate is divided over the traction, spindle, swivel and status channels, so the per-channel traction arrival rate is correspondingly lower, on the order of ~ 4 –5 Hz, an inter-sample gap of ~ 200 ms. The renderer reconciles this inter-sample gap with Kalman-based interpolation and dead-reckoning (Section 4.3). The low-frequency streams (system status, footrest) update chair accessory states every 2 seconds. Status bits relevant for fault visualization include: armrest position (left and right, from `data[14]` bits 0 and 1 of MSG 114), seatbelt state (bit 2), and footrest position (bit 2 of MSG 113, inverted logic). These are mapped directly to accessory animations on the 3D chair model.

These six messages are transported from the lift controller to the renderer through a five-stage pipeline. Each stage has a single responsibility: receive data from the previous stage, process it, and pass the result to the next. This separation makes it possible to develop and test each stage independently. Figure 1 shows the five layers.

Transport layer. Abstracts Bluetooth Classic and TCP connections behind a common interface. The Bluetooth transport communicates with an HC-06 module (a low-cost serial-to-Bluetooth adapter) on the lift controller. Before each session, 40 null bytes are sent to flush the module’s receive buffer. Message counters increment from 1 to 200 and wrap around.

Protocol parser. A finite state machine (FSM, a parser that moves through a fixed sequence of states for each incoming byte) [24] processes the incoming byte stream: `WAIT_START` $\rightarrow$ `MSG_NUMBER` $\rightarrow$ `COMMAND_ID` $\rightarrow$ `LENGTH` $\rightarrow$ `DATA` $\rightarrow$ `CHECKSUM`. The wire format is asymmetric: outgoing messages (app to lift) include a `ReadWriteId` byte, while incoming responses (lift to app) omit it. The byte layout is:

```
App  $\rightarrow$  Lift: [0x0A] [Nr] [Rw] [Cmd] [Len] [Data...] [Chk]
Lift  $\rightarrow$  App: [0x0A] [Nr] [Cmd] [Len] [Data...] [Chk]
```

The checksum covers all preceding bytes:

$$\text{Chk} = (0x0A + \text{MsgNr} + \text{CmdId} + \text{Len} + \sum \text{Data}_i) \bmod 256 \quad (1)$$

Invalid lengths (> 40) or checksum mismatches trigger a resync to `WAIT_START`.

Telemetry poller. Three concurrent loops (coroutines, i.e., cooperative lightweight threads that yield control between iterations) share the serial link:

- *Fast loop* (~ 20 Hz): alternates traction (MSG 110) and spindle (MSG 111) requests each frame tick.
- *Swivel loop*: polls the swivel angle via the `APP_TO_BUS` multiplexed channel (MSG 168). The AppToBus protocol uses a two-step handshake (send request; if the response is a single-byte ACK, poll for the actual data at

Table 3: Telemetry message types relevant for spatial visualization, classified along the three axes defined in Section 3.1.

Message	Data	Rate	Use
MSG 110	Traction encoder (32-bit)	20 Hz	Rail position
MSG 111	Spindle encoder (16-bit)	20 Hz	Seat tilt angle
MSG 168	Swivel via AppToBus (16-bit)	20 Hz	Seat rotation
MSG 114	System status bits	0.5 Hz	Arm/seatbelt state
MSG 113	Footrest status	0.5 Hz	Footrest position
MSG 44	Pilot state	On demand	Calibration run

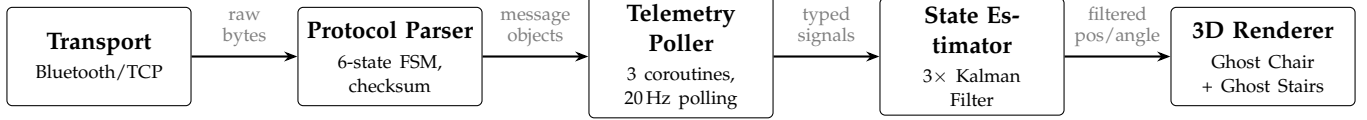


Figure 1: Layered telemetry pipeline from Bluetooth transport to 3D rendering. Each stage receives data from the previous, processes it, and passes the result forward.

100 ms intervals up to ten times). Successive swivel-polling cycles are separated by a 50 ms delay. Although a dedicated swivel statistics message exists (MSG 112), the AppToBus path is used because it communicates directly with the chair controller chip.

- *Slow loop* (0.5 Hz): polls system status (MSG 114) and footrest state (MSG 113) every 2 s.

Data parsing. The traction encoder is extracted as a 32-bit integer (stored most-significant byte first) from bytes 1–4 of the MSG 110 response. The spindle encoder is a 16-bit value from bytes 1–2 of MSG 111. The swivel angle is computed from MSG 168 as:

$$\theta_{\text{swivel}} = \frac{(\text{data}[3] \ll 8 \mid \text{data}[4]) - 0 \times 7FFF}{18} \quad (2)$$

The final stage of the pipeline is the 3D renderer, whose output is shown in Figure 2. The renderer composes the chair model, a generated stair geometry, and rail annotations from the filtered telemetry; the detailed visual design of these elements is part of the bereopsproduct rather than a research outcome of this article.

4.2 Encoder-to-Position Mapping

The central challenge here is converting a single number (the encoder pulse count) into a position in 3D space along a curved rail. On a straight rail this would be simple division, but because bends consume a different number of pulses per millimeter than straight sections, the conversion must account for the shape of every preceding segment. This section derives that conversion step by step.

4.2.1 Rail Geometry Construction

The lift controller stores the rail as a sequence of typed blocks: straight sections, vertical bends (up/down), and horizontal bends (left/right), each measured via incremental encoders [6]. Each block specifies a type and an amount. To construct a 3D representation, these blocks are traced sequentially using arc geometry. Figure 3 shows the three segment types and their arc-tracing geometry.

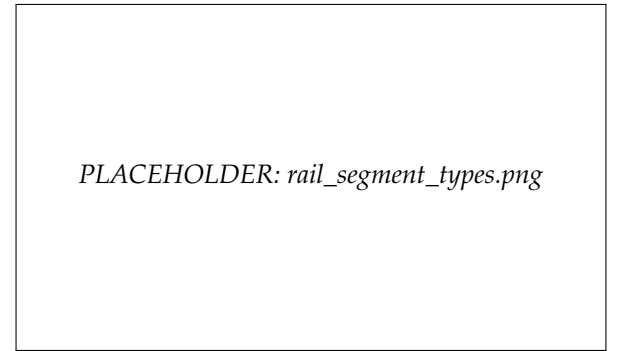


Figure 3: The three rail segment types: straight (left), vertical bend (center), and horizontal bend (right), with their respective arc-tracing geometry and encoder pulse costs.

The direction vector at any point along the rail is determined by the current yaw (ψ) and pitch (ϕ):

$$\mathbf{d}(\psi, \phi) = \begin{pmatrix} -\sin \psi \cos \phi \\ \sin \phi \\ -\cos \psi \cos \phi \end{pmatrix} \quad (3)$$

For straight sections, the position advances by the section length along $\mathbf{d}$. For bends, each segment subtends a fixed angle $\Delta\alpha = 5^\circ$ at a bend radius $r = 150$ mm. The arc length per segment is:

$$s = r \cdot \Delta\alpha_{\text{rad}} = 0.150 \cdot \frac{5\pi}{180} \approx 0.01309 \text{ m} \quad (4)$$

Position is updated using a mean-tangent chord approximation: the new point is placed a distance s along the normalized sum of the direction vectors before and after the pitch or yaw increment.

$$\mathbf{p}_{i+1} = \mathbf{p}_i + s \cdot \frac{\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}}{\|\mathbf{d}_{\text{before}} + \mathbf{d}_{\text{after}}\|} \quad (5)$$

This is a second-order scheme: a Taylor expansion of the exact circular chord $2r \sin(\Delta\alpha/2)$ against the approximation $s = r\Delta\alpha$ gives a per-segment displacement error of

$$|r\Delta\alpha - 2r \sin(\Delta\alpha/2)| = \frac{r\Delta\alpha^3}{24} + O(\Delta\alpha^5). \quad (6)$$

For the parameters used here ($r = 150$ mm, $\Delta\alpha = 5^\circ$), this evaluates to $\approx 4.2 \mu\text{m}$ per segment, which accumulates to < 1 mm over an 8-bend rail of 144 segments.



Figure 2: The pipeline output: the 3D ghost chair follows the rail based on live telemetry, with semi-transparent ghost stairs providing spatial context and rail annotations identifying segment boundaries.

The term is therefore not a plausible dominant source of the bend-transition errors reported in Section 4.4; see the discussion in Section 5. The term “trapezoidal” rule is avoided here because, unlike the scalar trapezoidal rule of [8, §3.6], Equation 5 advances a point in a direction that is the renormalized average of two unit tangents, which is geometric rather than arithmetic [16, 15].

4.2.2 Piecewise Encoder Mapping

Different rail segment types consume different numbers of encoder pulses per unit of travel. The firmware defines the following constants:

$$\begin{aligned} k_{\text{straight}} &= 3.361352398 \text{ pulses/mm} \\ k_{\text{up}} &= 37.54666 \text{ pulses/segment} \\ k_{\text{down}} &= 50.45333 \text{ pulses/segment} \\ k_{\text{horiz}} &= 44.0 \text{ pulses/segment} \\ E_{\text{start}} &= 100 \text{ pulses (initial offset)} \end{aligned} \quad (7)$$

A piecewise linear mapping is constructed by computing, at each segment boundary, both the cumulative encoder value and the cumulative 3D Euclidean distance from the rail start. For an encoder reading E , the normalized position $t \in [0, 1]$ is found by:

1. Locating the segment $[E_i, E_{i+1}]$ containing E .
2. Linearly interpolating the 3D distance: $d = d_i + (d_{i+1} - d_i) \cdot \frac{E - E_i}{E_{i+1} - E_i}$
3. Normalizing: $t = d/d_{\text{total}}$

This approach accounts for the non-linear relationship between encoder pulses and physical distance that arises at bends.

4.2.3 Spindle Angle Lookup

The spindle encoder value is converted to a seat tilt angle using a 751-entry lookup table ported from the firmware’s

Curve.h. The table maps angles from 0.0° to 75.0° in 0.1° increments to corresponding encoder pulse counts. Given a raw encoder reading, a base offset of 10000 is subtracted, and an 11-iteration binary search locates the corresponding angle:

$$\theta_{\text{spindle}} = \frac{\text{bisect}(\text{TABLE}, \max(0, E_{\text{raw}} - 10000))}{10} \quad (8)$$

4.3 Mathematical Models for State Estimation

The sensor readings arrive roughly every ~ 200 ms per channel (see Section 4.1), but the display must update every 17 ms (60 frames per second). The models in this section solve three problems: (1) estimating a continuous velocity from the discrete pulse-counter stream so that motion can be interpolated between samples, (2) predicting where the chair is between measurements so the display stays fluid, and (3) computing the chair’s 3D orientation (which way it faces, how it tilts) from separate sensor streams without mathematical errors that would cause the model to “snap” or distort.

4.3.1 Kalman Filter

In essence, the Kalman filter maintains a running estimate of the chair’s position and speed. Each time a new sensor reading arrives, it blends the prediction (based on the previous position and speed) with the new measurement. The result is a continuously updated position-and-velocity state that the render loop can extrapolate between sensor samples, so the chair moves fluidly at the display frame rate even though per-channel updates only arrive at ~ 4 –5 Hz.

Formally, a 2D constant-velocity Kalman filter [11, 12] is applied independently to each telemetry axis (traction position, spindle angle, swivel angle). The state vector is

$\mathbf{x} = [p, v]^T$ (position and velocity).

Prediction step (time update with interval Δt):

$$\begin{aligned} p &\leftarrow p + v \cdot \Delta t \\ P_{00} &\leftarrow P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + \sigma_a^2 \cdot \frac{\Delta t^3}{3} \\ P_{01} &\leftarrow P_{01} + \Delta t \cdot P_{11} + \sigma_a^2 \cdot \frac{\Delta t^2}{2} \\ P_{11} &\leftarrow P_{11} + \sigma_a^2 \cdot \Delta t \end{aligned} \quad (9)$$

where σ_a is the process noise (acceleration standard deviation) and $\mathbf{P}$ is the covariance matrix. The process noise matrix is that of the continuous white-noise-acceleration model for a constant-velocity state [25, 26], in which the unmodelled acceleration is treated as a zero-mean white-noise process of intensity σ_a^2 :

$$\mathbf{Q} = \sigma_a^2 \begin{bmatrix} \Delta t^3/3 & \Delta t^2/2 \\ \Delta t^2/2 & \Delta t \end{bmatrix} \quad (10)$$

Measurement update (correction with measurement z):

$$\begin{aligned} y &= z - p \\ S &= P_{00} + \sigma_m^2 \\ K_0 &= P_{00}/S \\ K_1 &= P_{01}/S \\ p &\leftarrow p + K_0 \cdot y \\ v &\leftarrow v + K_1 \cdot y \\ P_{11} &\leftarrow P_{11} - K_1 \cdot P_{01} \\ P_{00} &\leftarrow (1 - K_0) P_{00} \\ P_{01} &\leftarrow (1 - K_0) P_{01} \end{aligned} \quad (11)$$

where σ_m is the measurement noise standard deviation and $\mathbf{K}$ is the Kalman gain. The last three lines propagate the covariance through the correction as $\mathbf{P} \leftarrow (\mathbf{I} - \mathbf{K}\mathbf{H}) \mathbf{P}$ with the scalar position-measurement model $\mathbf{H} = [1 \ 0]$, written out componentwise and ordered so that each assignment uses the pre-update entries. This closes the recursion: the prediction step (Equation 9) grows the covariance while the correction shrinks it, so the gain $\mathbf{K}$ adapts to the current estimate uncertainty [12].

The tuning parameters per axis are listed in Table 4. The σ_a (process noise / acceleration uncertainty) and σ_m (measurement uncertainty) values were determined empirically by iterating on observed filter output during test runs. Because the traction encoder is a digital pulse-counter rather than an analog noisy sensor, σ_m functions here as a smoothness/responsiveness knob between sample points rather than as a literal measurement-noise standard deviation; σ_a controls how aggressively dead-reckoning extrapolates between samples. The consequences of this empirical-rather-than-derived tuning for generalisability are discussed in Section 5.

4.3.2 Dead-Reckoning Extrapolation

Between sensor updates (which arrive every ~ 200 ms per channel), the display still needs to move the chair smoothly every frame (every ~ 17 ms). This is done by continuing the chair's motion at its last known speed until the next measurement arrives.

Between measurement updates, the display position is extrapolated from the Kalman filter state using dead reckoning [27]:

$$p_{\text{target}} = p_{\text{KF}} + v_{\text{KF}} \cdot t_{\text{elapsed}} \quad (12)$$

After 0.8 s without a measurement, velocity decays exponentially:

$$v \leftarrow v \cdot e^{-5.0 \cdot (t_{\text{elapsed}} - 0.8)} \quad (13)$$

The display position is then smoothed with a frame-rate-independent exponential filter. This blend is the exact discrete-time solution of the first-order relaxation $\dot{p} = \alpha(p_{\text{target}} - p)$ over a frame interval Δt , which integrates to $p \leftarrow p_{\text{target}} + (p - p_{\text{target}}) e^{-\alpha \Delta t}$, equivalently:

$$p_{\text{display}} = \text{lerp}(p_{\text{display}}, p_{\text{target}}, 1 - e^{-\alpha \Delta t}) \quad (14)$$

where α is the smoothing rate from Table 4. Writing the blend factor as $1 - e^{-\alpha \Delta t}$ rather than a fixed constant keeps the smoothing time-constant independent of the (variable) render frame interval.

4.3.3 Catmull-Rom Spline Tangent

The rail is defined by a series of 3D points. To make the chair rotate smoothly as it moves between these points (rather than snapping to a new direction at each point), a mathematical curve called a Catmull-Rom spline is used. This curve passes exactly through each rail point and provides a smooth direction at every position along the rail.

The tangent $\mathbf{T}(t) = \frac{d}{dt}\mathbf{C}(t)$ at parameter t is obtained by differentiating the uniform Catmull-Rom position polynomial [14] over the surrounding control points $\mathbf{p}_0 \dots \mathbf{p}_3$, which yields:

$$\mathbf{T}(t) = \frac{1}{2} [(-\mathbf{p}_0 + \mathbf{p}_2) + 2(2\mathbf{p}_0 - 5\mathbf{p}_1 + 4\mathbf{p}_2 - \mathbf{p}_3)t + 3(-\mathbf{p}_0 + 3\mathbf{p}_1 - 3\mathbf{p}_2 + \mathbf{p}_3)t^2] \quad (15)$$

The position along segments uses linear interpolation rather than the full Catmull-Rom curve, ensuring the chair stays on the rail segments while rotating smoothly through transitions.

4.3.4 Quaternion Orientation

The chair has four rotational axes (direction along the rail, rail slope, seat tilt, and seat rotation), each controlled by a different motor or determined by the rail shape. Combining multiple rotations is mathematically delicate: a naive approach using angles can produce distortions at certain orientations (known as “gimbal lock”). Quaternions are a mathematical representation of rotations that avoids this problem entirely [17].

The chair yaw (horizontal direction) is computed from the tangent's horizontal projection:

$$\psi = \text{atan2}(T_x, T_z) \quad (16)$$

with phase unwrapping to avoid 2π discontinuities.

Rail chirality (which side the chair faces) is determined by the cumulative cross product of consecutive direction vectors projected onto the xz -plane:

$$\mathbf{C} = \sum_i (\mathbf{d}_x^i \cdot \mathbf{d}_z^{i+1} - \mathbf{d}_z^i \cdot \mathbf{d}_x^{i+1}) \quad (17)$$

Table 4: Kalman filter tuning parameters per telemetry axis.

Parameter	Traction	Spindle	Swivel
σ_a (process noise)	0.03	8.0	10.0
σ_m (measurement noise)	0.0002	0.2	0.5
Display smoothing rate	8.0	12.0	12.0
Max velocity	$1.2v_{\max}$	$30^\circ/\text{s}$	$40^\circ/\text{s}$

If $C \geq 0$, the rail is left-turning and the chair faces outward at $+90^\circ$; otherwise at $+270^\circ$.

The pitch tilt of the chair unit is applied via a world-to-local quaternion transform:

$$q_{\text{unit}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a}_{\text{pitch}}, -\phi) \cdot q_{\text{parent}} \quad (18)$$

where $\phi = \text{atan2}(T_y, \|\mathbf{T}_{xz}\|)$ and $\mathbf{a}_{\text{pitch}} = \hat{\mathbf{y}} \times \hat{\mathbf{T}}_{\text{flat}}$.

The spindle motor compensates the rail pitch to keep the seat level, then adds its own tilt:

$$q_{\text{spindle}} = q_{\text{parent}}^{-1} \cdot q(\mathbf{a}, -\phi + \theta_{\text{spindle}}) \cdot q_{\text{parent}} \quad (19)$$

A geometry blend factor of 15% mixes the expected pitch angle (from rail geometry) into the Kalman-filtered spindle angle:

$$\theta_{\text{target}} = 0.85 \cdot \theta_{\text{KF}} + 0.15 \cdot \phi_{\text{rail}} \quad (20)$$

The 0.85/0.15 split is the value currently configured in the implementation (`SPINDLE_GEOMETRY_WEIGHT = 0.15`). It is not derived from a first-principles noise model, so its sensitivity to encoder noise and rail geometry is not formally bounded; this limitation is reported in Section 5.

4.4 Validation Evidence

This section reports the empirical evidence for the DV1–DV3 criteria that require physical measurement. The prototype is validated across two measurement campaigns on physical stairlift hardware: a first campaign on a 3403 mm rail (11 rail-parts, one 90° horizontal bend; two calibration runs bottom-to-top and top-to-bottom, preceded by a 13 s stationary baseline), and a second campaign on a longer 5428 mm rail (eight straight sections and seven bends: two horizontal and five vertical). The second campaign additionally includes a dedicated continuous-drive log for the velocity-based DV3(b) test and two static orientation tests (spindle and swivel). The system polls telemetry at an aggregate ~ 10 Hz of round-trips during the runs (shared across all channels, Section 4.1) and logs every event with millisecond timestamps via the in-app TestLogger module.

Measurement methodology. Reference markers are placed at known offsets on straight rail-parts before the runs (five on the 3403 mm rail; five on the 5428 mm rail, on rail-parts rd3, rd5, rd7, rd9, and rd13). Each marker is captured in both drive directions, giving ten marker readings per rail. Physical chair position is read with a steel tape (graduation 1 mm, reading uncertainty $\approx \pm 1$ mm). Position accuracy is evaluated at the two levels prescribed by the DV2(b) design:

- *Display vs. physical* (primary): the displayed position is compared against the physical marker position, taken

as the firmware-encoded start of the marker’s rail-part plus the tape-measured offset within that part. Referencing each marker to its rail-part origin captures the end-to-end within-part accuracy a service engineer would see, including the rail-part’s own encoded-vs-physical scaling.

- *Display vs. firmware-encoded* (secondary): the displayed position is compared against the firmware-encoded position derived from the same encoder reading via the calibration map (Section 4.2). This isolates the pipeline-integration error; the difference between the two measures is exactly the upstream rail-configurator contribution, reported explicitly rather than deferred.

Bluetooth round-trip time is measured per request/response pair by the application layer (5295 round-trips on the first rail, 5872 on the second). The encoder baseline is measured during the 13 s stationary window. For DV3(b), the rendered position is logged at frame rate during a continuous drive without intermediate stops, and the per-frame velocity $v[i] = (p[i+1] - p[i]) / \Delta t[i]$ is analysed for spikes exceeding $5 \times$ the median $|v|$.

PLACEHOLDER: calibration_run.png

Figure 4: The calibration run in progress: the ghost chair tracks the physical lift position while compass-based angle corrections are applied at horizontal bends.

Calibration run protocol. Figure 4 shows a calibration run in progress. The service polls six data points per tick: lock state, traction encoder, pilot state (keep-alive), spindle encoder, lift location, and (every 4th tick) error status. Speed is managed automatically via the firmware’s speed-setting register (percent of the lift’s configured maximum speed, as defined in MPLAB): the protocol writes 50 % (normal) for straight sections and 20 % (reduced) for bends and for short straights under 250 mm.

Angle correction. At horizontal bends, the compass tracker (running on the Android tablet, reading its gyroscope) measures accumulated yaw via gyroscope integration. The compass subsystem works in degrees throughout (matching the firmware’s segment-angle convention of 5° per bend-segment), so the gyroscope rate ω_z (in rad/s, as delivered by the Android sensor API) is con-

verted to deg/s before integration:

$$\psi_{\text{acc}} += \omega_z \cdot \frac{180}{\pi} \cdot \Delta t \quad (\text{deg}) \quad (21)$$

The correction is computed as:

$$\delta = |\theta_{\text{bend}}/2| - |\psi_{\text{measured}}| \quad (22)$$

where $\theta_{\text{bend}} = n_{\text{segments}} \cdot 5^\circ$ is the expected total bend angle (distinct from the spindle target angle θ_{target} in Equation 20). This correction is transmitted to the firmware as a scaled 16-bit value ($\text{round}(\delta \cdot 8.0)$).

Split-bend compensation. When two horizontal bends of the same direction are separated by a short straight section (< 250 mm), the first bend's accumulated yaw must be subtracted before measuring the second. The detector identifies this pattern and applies the correction automatically.

Quantitative evidence per criterion. Table 5 collects the per-rail measurements bearing on DV1(b) and DV2(b); DV3(b) and the orientation tests are measured on dedicated runs and reported in the text below.

DV1(b): latency. Bluetooth round-trip latency met the pre-registered percentile thresholds on both rails: P95 was 113 ms (rail 1) and 112 ms (rail 2), and P99 was 135 ms and 134 ms, against the 150 ms and 250 ms thresholds (means 89 ms and 88 ms over 5295 and 5872 round-trips). Figure 5 shows the distribution for the second campaign: the bulk of round-trips cluster near 80–90 ms and the whole distribution lies below the 150 ms P95 limit apart from a thin tail (a single 167 ms maximum on rail 2; one 543 ms outlier on rail 1). Because the Kalman filter and dead-reckoning mask single-sample latency at the display side, this latency does not surface as visible lag; the percentile bound additionally keeps round-trip gaps within the 0.8 s horizon for which the dead-reckoning extrapolation is designed (Section 4.3).

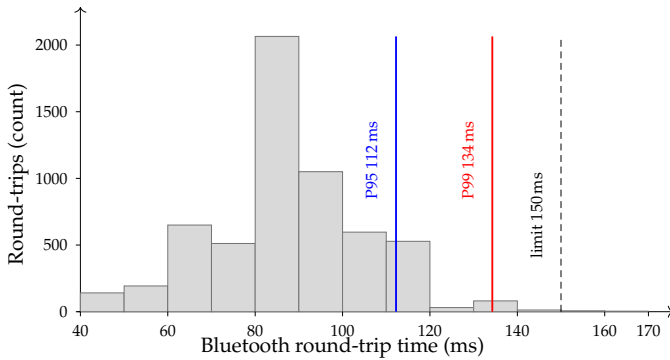


Figure 5: Distribution of Bluetooth round-trip times over campaign 2 ($n=5872$, 10 ms bins). The pre-registered DV1(b) thresholds are P95 < 150 ms and P99 < 250 ms; the measured P95 (112 ms) and P99 (134 ms) both clear them with margin, and the full distribution lies below the 150 ms limit except for a thin tail (max 167 ms).

DV2(b): position accuracy. The two measurement levels separate the pipeline from the upstream rail-configurator. The pipeline-integration error (display vs. firmware-encoded position) was at or below 0.7 mm on rail 1 (mean 0.39 mm, SD 0.22 mm, 95 % CI of the mean [0.23, 0.55] mm; t-based, $df=9$) and ≤ 0.1 mm on rail 2, both below the $\approx \pm 1$ mm tape resolution, confirming that

the encoder-to-position math reproduces the firmware position essentially exactly. The end-to-end display-vs-physical error, measured on the 5428 mm rail (Figure 6), reached a maximum of 9.88 mm at marker rd7, inside the 10 mm tolerance for rails longer than 5 m, so the primary DV2(b) criterion passes. All five errors are negative (mean -6.9 mm): the displayed position reads consistently a few millimetres short of physical. The ≈ 7 mm gap between this display-vs-physical figure and the near-zero pipeline-integration error is the rail-configurator contribution made explicit (below), not a pipeline error. Direction repeatability was high: the same physical marker read in both drive directions agreed to within 4.1 mm across all five markers.

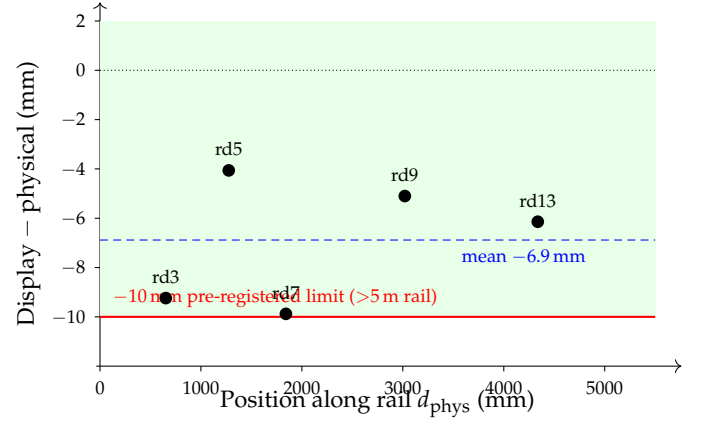


Figure 6: Display-vs-physical position error at each reference marker on the 5428 mm validation rail (campaign 2). Each marker is referenced to the firmware-encoded start of its rail-part plus a tape-measured offset within that part (Section 4.4). All five errors fall inside the pre-registered -10 mm tolerance for rails longer than 5 m; the consistently negative sign (mean -6.9 mm) reflects the rail-configurator's encoded-vs-physical contribution, isolated from the near-zero pipeline-integration error (Table 5).

Rail-configurator contribution. An independent tape measurement of the first rail showed the firmware-encoded rail-part lengths deviating from the physically measured lengths by up to 16 mm per part (rd1: +16 mm, rd5: -9 mm, rd11: -10 mm; net +10 mm over the 3.4 m rail, 0.3%). This is a property of the rail-configurator's data acquisition during lift installation, not of the visualization pipeline, and it is the most plausible source of the systematic display-vs-physical offset on rail 2: because the displayed position tracks the firmware-encoded geometry essentially exactly (the ≤ 0.1 mm pipeline error), any encoded-vs-physical scaling of a rail-part carries into the end-to-end error. This configurator error was quantified independently on rail 1 only; it was not separately measured on rail 2, and the rail-2 error was not decomposed into within-part tracking versus encoded-length error (which would require physically measured *cumulative* distances to each rail-part start, not collected here). The primary DV2(b) figure therefore includes this upstream contribution by design (it is the error a service engineer would see), while the secondary measure isolates the pipeline.

DV3(b): smoothness. The velocity-based smoothness test

Table 5: Validation measurements across the two campaigns. Position is evaluated at two levels: the pipeline-integration error (display vs. firmware-encoded position from the same encoder reading) isolates the pipeline, while the display-vs-physical error is the end-to-end accuracy referenced per rail-part (see methodology above). An “n/a” marks a quantity not collected for that rail. Pre-registered passes are stated in the rightmost column and discussed in Section 5.

Metric	Rail 1 (3403 mm)	Rail 2 (5428 mm)	Bears on / threshold
Geometry	11 parts, 1 horiz. bend	8 straight, 7 bends (2 horiz., 5 vert.)	n/a
Reference markers (B→T + T→B)	10	10	n/a
<i>DV1(b): Bluetooth round-trip latency</i>			
mean (n)	89 ms (5295)	88 ms (5872)	informational
P95	113 ms	112 ms	< 150 ms (pass)
P99	135 ms	134 ms	< 250 ms (pass)
max outlier	543 ms	167 ms	n/a
<i>DV2(b): position accuracy</i>			
Pipeline-integration error, max	0.7 mm	≤ 0.1 mm	secondary (pipeline isolation)
mean (n=10)	0.39 mm	≈ 0.0 mm	secondary
Display-vs-physical error, max	n/a	9.9 mm	primary: < 10 mm (>5 m) (pass)
mean (signed, n=5)	n/a	−6.9 mm	primary
Direction repeatability, max	n/a	4.1 mm	n/a
Continuous drive without crash	524 s	253 s	O1 (pipeline robustness)
Stationary baseline pulse variation	0 (const. 16030)	n/a	sensor jitter at rest

was run on a dedicated continuous drive (no intermediate stops, near-full rail) on the 5428 mm rail, logging the rendered position at frame rate (13 596 frames, ~53 fps on the tablet). On the steady bottom-to-top leg, velocity spikes exceeding $5 \times$ the median $|v|$ occurred on 0.04 % of frames for the traction axis (0.04 % spindle, 0.21 % swivel). For reference, a last-value-hold (staircase) rendering (the failure mode the criterion is designed to detect) would produce roughly one spike per ten frames (~10 %), so the observed rate is more than two orders of magnitude (a factor of ~250) below that signature: no staircase is present. The few isolated spikes occur at normal frame intervals (the single 153 ms render hitch in the log produced no position spike, the dead-reckoning having bridged it) and are mild speed variations rather than a stepped pattern. A desktop precursor run (60 fps, logged 2026-06-02, before the 2026-06-04 pre-registration and so corroborating rather than confirmatory) gave 0, 0, and 1 spike on the three axes, the same behaviour without the tablet’s lower, more variable frame rate; render frame rate is a separate performance property and does not enter the velocity-based smoothness measure (Section 3.3).

Orientation accuracy (spindle, swivel). Two static tests checked the displayed seat orientation against external references. For the spindle (seat-tilt) axis, the displayed lookup angle was compared against a digital spirit level over the full 0–75° range in 15° steps: the maximum deviation was 1.05° (at the 75° mechanical end stop), mean ≈ 0.4°. For the swivel axis, the displayed angle was compared against the tablet gyroscope over ±60°: the maximum deviation was 2.83°, within the ±3.5° encoder-to-physical tolerance, and ~2.6° of the spread is attributable to gyroscope drift in the reference itself (the gyro read 2.6° at a commanded 0° after the sweep). Both axes are seat-mounted and rail-independent, so a single test characterises them across configurations. On level track the spindle render deliberately shows $0.85 \times$ the lookup angle (the 15 % geometry blend, Equation 20, with $\phi_{\text{rail}} \approx 0$); the validation therefore compares against the lookup an-

gle, not the blended render value, so that the blend does not appear as a spurious ~15 % error.

Pulse-encoder behaviour at rest. The 13 s stationary baseline shows zero variation in the raw encoder reading (constant 16030 over the window): the traction sensor produces no value-jitter on the sample itself while the chair is stationary, which is precisely why the original “jitter ≥ 10× smaller than raw” criterion is undefined for this sensor and was replaced by the velocity-based DV3(b) test.

Logged events. No Bluetooth disconnects, parser checksum errors, or manual interventions occurred during any run. The latency outliers (543 ms on rail 1, 167 ms on rail 2) produced no visible position jumps. The interpretation of all measurements against the DV1(b)–DV3(b) criteria is given in Section 5.

5 Conclusion

5.1 Answers to Research Questions

Pipeline architecture. The classification of 145 firmware message types revealed that only six carry spatially interpretable information, so the bandwidth-constrained Bluetooth link does not need to carry the full telemetry stream. By targeting only these six messages with a multi-rate polling strategy, the pipeline keeps the position-relevant channels polled at the highest rate the shared serial link sustains (an aggregate ~10 Hz of round-trips, Section 4.4); future extensions (e.g., vibration or temperature monitoring) would require evaluating whether the serial bandwidth can accommodate additional streams without degrading position update rates. Over the two campaigns the Bluetooth round-trip latency met the pre-registered DV1(b) percentile thresholds on both rails: P95 was 113 ms (rail 1) and 112 ms (rail 2) against the 150 ms bound, and P99 was 135 ms and 134 ms against the 250 ms bound (means 89 ms and 88 ms; Table 5, Figure 5), so

DV1(b) passes quantitatively. The visible end-user property was additionally observed to hold (O3, qualitatively): the chair on screen tracks the physical chair without perceptible delay, consistent with the Kalman filter and dead-reckoning masking single-sample latency at the display side rather than passing it through; a quantitative video frame-offset measurement of this lag is left to future work (Section 5, Recommendations). The five-layer architecture itself isolates failure modes by design: byte-level errors are caught by the protocol parser’s checksum, while higher-level issues (connection loss, timeout) are handled by the transport layer; across the continuous drives of both campaigns the pipeline sustained operation without crashes or UI corruption (O1, Table 5). Robustness under adversarial fault injection was not exercised and is reported as a limitation.

Encoder mapping. The piecewise mapping resolves the core geometric problem identified in Section 2.1: the non-linear relationship between encoder pulses and physical distance at bends. The primary DV2(b) criterion (display vs. physical position) was met on the 5428 mm rail, where the maximum end-to-end error was 9.9 mm against the 10 mm tolerance for rails longer than 5 m (Figure 6). Isolating the pipeline shows where that error originates: the pipeline-integration error (display vs. firmware-encoded position) was ≤ 0.7 mm on the first rail (mean 0.39 mm, SD 0.22 mm, 95 % CI of the mean [0.23, 0.55] mm) and ≤ 0.1 mm on the second, in every case below the $\approx \pm 1$ mm tape resolution, i.e. the mapping reproduces the firmware position essentially exactly. Equation 6 bounds the per-segment arc-tracing error at $\approx 4.2 \mu\text{m}$ per segment, consistent with this sub-millimetre residual and with the finite precision of the firmware pulse-cost constants (Equation 7). The remaining ~ 7 mm of the display-vs-physical error is therefore most plausibly attributable to the rail-configurator’s encoded-vs-physical discrepancy, independently measured on rail 1 at up to 16 mm per rail-part (net +10 mm over the 3.4 m rail, 0.3 %), rather than to the pipeline, since the display tracks the firmware geometry exactly; this is consistent with the systematically negative rail-2 errors (mean -6.9 mm: the displayed position reads short wherever the encoded rail-part is shorter than physical). A direct rail-2 quantification of this split was not made. This upstream source is discussed further under Limitations.

Mathematical models. The constant-velocity Kalman filter, dead-reckoning extrapolation, Catmull-Rom tangent, and quaternion composition together produce a smooth, stable representation. The reformulated DV3(b) criterion is now evaluated quantitatively: on a dedicated continuous drive over the 5428 mm rail, velocity spikes exceeding $5\times$ the median frame-to-frame $|v|$ occurred on 0.04 % of frames (traction axis), more than two orders of magnitude below the ~ 10 % spike rate a last-value-hold rendering would produce, so no staircase is present and DV3(b) is substantively met (the literal “0 spikes” threshold is narrowly exceeded; see Limitations). The 13 s stationary baseline showed zero pulses of encoder variation (constant 16030), which is why the original jitter-ratio criterion was undefined for this digital pulse-counter and was replaced by the velocity test. The relevant role of the Kalman filter in this pipeline is therefore not noise-reduction but *temporal interpolation*: gap-filling between

sensor samples for the render loop and velocity estimation for dead-reckoning during latency spikes. The orientation models were checked against external references in two static tests: the spindle seat-tilt tracked a digital spirit level to within 1.05° over $0-75^\circ$, and the swivel angle tracked the tablet gyroscope to within 2.83° over $\pm 60^\circ$ (inside the $\pm 3.5^\circ$ tolerance, part of the spread being drift in the gyroscope reference). The Kalman tuning parameters (Table 4) and the 0.85/0.15 spindle-pitch blend remain configured values rather than derivations from a first-principles noise model; sensitivity to these choices is not formally bounded and is reported as a limitation.

5.2 Answer to Central Question

The data processing pipeline required to transform live stairlift telemetry into a stable, accurate 3D visualization consists of five layers: wireless transport, checksummed protocol parsing, multi-rate coroutine polling, recursive state estimation, and spline-interpolated 3D rendering. The mathematical foundation rests on three pillars: piecewise encoder mapping for non-linear rail geometries, a constant-velocity recursive filter for temporal interpolation and inter-measurement prediction between samples, and quaternion-based orientation reconstruction from spline tangent vectors.

Across the two validation campaigns the implemented pipeline and models meet the position-accuracy and latency criteria and substantively pass the smoothness criterion. Position accuracy (DV2(b)) passes at both levels: the pipeline-integration error stays ≤ 0.7 mm on both rails, and the end-to-end display-vs-physical error on the 5428 mm rail is at most 9.9 mm, inside the 10 mm tolerance for rails longer than 5 m. Latency (DV1(b)) meets the percentile thresholds on both rails ($P95 \leq 113$ ms, $P99 \leq 135$ ms). Smoothness (DV3(b)) shows no staircase (velocity spikes on 0.04 % of frames against ~ 10 % for a last-value-hold), so the substantive criterion is met, though the literal pre-registered “0 spikes” threshold is narrowly exceeded by a few isolated, mild spikes (see Limitations). The full per-criterion outcome is collected in Table 5. The latency and pipeline-integration measures span both rail configurations (3403 mm with one horizontal bend; 5428 mm with two horizontal and several vertical bends); the primary end-to-end display-vs-physical measure was collected on the 5428 mm rail only. This two-geometry coverage strengthens the case for generalisation beyond a single rail, but broader generalisation across the full range of rail designs (and the primary position measure on more rails) would still require further runs and is left to future work.

The implementation that realises this pipeline (the Godot renderer, the Bluetooth/TCP transport and protocol parser, the multi-rate telemetry poller, and the per-axis state estimator) is the accompanying *beroeepsproduct*. Its full source code is supplied as a separate archive with this report and is available for inspection at DeVi Comfort; the equations and algorithms reported here (the piecewise encoder map, the Kalman predict/correct steps, the dead-reckoning extrapolation, and the quaternion composition) are the research instruments implemented in that code.

5.3 Limitations

- **Sample size.** The validation spans two rail configurations (3403 mm and 5428 mm), each measured in a single session. This is stronger than a single rail but still far from the full range of rail designs; additional installations, and repeated sessions per rail, would be needed for statistical generalisation, and the per-marker counts (five per rail) are small.
- **The DV1(b) and DV3(b) criteria were revised after the first campaign.** Their original (v1) forms (a single-sample 100 ms round-trip target and a “Kalman jitter $\geq 10\times$ smaller than raw” ratio) assumed behaviour this system does not exhibit (latency is masked at the display, and a digital pulse-encoder shows zero value-jitter at rest; the 13 s baseline confirmed this). They were reformulated into the quantitative v2 forms (percentile latency, velocity-based smoothness) and pre-registered with a version-control timestamp before the second campaign. The decisive longer-rail runs are therefore scored against fixed-in-advance thresholds, but for the *first* campaign these criteria are post-hoc; only the second campaign is a true pre-registered test of DV1(b) and DV3(b). For DV2(b), only the pipeline-integration (display-vs-firmware) level was collected in the first campaign; the v2 pre-registration restored display-vs-physical as the primary measure, collected in the second campaign (hence the “n/a” for rail 1 in Table 5).
- **Display-vs-physical referencing.** The primary DV2(b) measure references each marker to the firmware-encoded start of its rail-part plus a tape-measured offset *within* that part. This captures within-part end-to-end accuracy, but not cross-part cumulative drift in the configurator geometry; bounding the absolute displayed position over the whole rail would require physically measured cumulative distances to each rail-part, which were not collected. The reported maximum (9.9 mm) is thus a within-part figure.
- **Smoothness passed at 0.04 %, not a literal zero.** DV3(b) is written as “0 spikes”; the measured rate is 0.04 % of frames (a few isolated, mild speed variations, more than two orders of magnitude below the staircase signature). The test was run once per platform, and the return leg of the continuous drive was less uniform (60 % of frames moving versus 95 %, over a 156 s versus 94 s window, with the swivel axis moving only at the two horizontal bends), which depressed the median $|v|$ and inflated the relative spike rate there (swivel 1.21 %, 17 spikes; traction 0.06 %; whole-window swivel 0.78 %); the steady bottom-to-top leg is therefore taken as representative.
- **Render frame rate on target hardware.** The tablet sustained $\sim 42\text{--}53$ fps during the runs, below the 60 fps of the desktop precursor, with occasional hitches (one 153 ms frame). This is a rendering-performance property, separate from the frame-rate-independent velocity smoothness measure (which passed), but it bears on perceived fluidity on lower-end hardware.
- **Rail-configurator precision dominates the end-to-end error.** The display-vs-physical error (max 9.9 mm, mean -6.9 mm) is dominated by the rail-configurator’s encoded-vs-physical discrepancy (up to 16 mm per rail-part), not by the pipeline (≤ 0.1 mm). The configurator

is treated as a given input to the pipeline (Section 3.1); a dedicated study with an external reference (laser distance meter) would characterise and potentially correct this upstream source.

- **Empirical filter tuning.** The Kalman tuning parameters and the 0.85/0.15 spindle blend ratio are configured values in the implementation rather than derivations from a measured sensor noise model. A formal sensitivity analysis is absent.
- **Bend-transition error not decomposed.** Reference markers are placed on straight rail-parts; position error at the bends themselves is not measured directly and cannot be attributed individually to the firmware pulse-cost constants, mechanical backlash, or the arc-tracing scheme without per-component instrumentation. Equation 6 rules out the arc-tracing scheme as a dominant source at the present segment resolution, but the remaining sources are not measured separately.
- **Orientation references are mixed.** The spindle seat-tilt is validated against an external digital spirit level (max 1.05°). The swivel angle, however, is validated only against the tablet gyroscope, which drifts ($\sim 2.6^\circ$ over a sweep), so its absolute accuracy is bounded by the reference rather than fully established; and the compass-based bend yaw-correction is generated by, and reported relative to, that same gyroscope, so it remains an internal-consistency check. A drift-free protractor template would tighten the swivel and correction figures. Unlike DV1(b)/DV3(b), the spindle ($\sim 1\text{--}2^\circ$) and swivel ($\pm 3.5^\circ$) pass thresholds were not crisply pre-registered before their 2026-06-04 runs (the pre-registration listed them as “to be confirmed”), and the $\pm 3.5^\circ$ figure is the inherent encoder-to-physical tolerance used as the bound rather than an independently fixed target; the orientation pass should therefore be read as a coarse no-gross-deviation check, not a strictly pre-registered test.
- **Self-validation.** The author defined the pass/fail criteria, executed the runs, and assigned the pass/fail labels. Criteria were pre-registered before the second campaign, but no independent second observer scored the results. Such independent scoring was foreseen in the measurement design (an explicitly recommended extra in the pre-registration) and was dropped only for time, so this is a known, pre-identified gap; for the first campaign, whose criteria are post-hoc, the single-author labelling also applies.
- **Robustness not tested adversarially.** Byte-level corruption handling by the protocol parser is supported by the architecture but was not exercised under fault injection; the observation that no corruption occurred during the runs is not equivalent to a robustness test.

5.4 Recommendations

- **Rail-configurator precision study.** The observation that firmware-encoded rail-part lengths deviated from physical tape measurements by up to 16 mm per part (Section 4.4, Limitations) is a candidate upstream contributor to end-to-end position error. A dedicated study that traces a rail through the configurator workflow with an external reference (laser distance meter) would quantify this source separately from the pipeline-integration error reported here.

- **High-speed video for visible lag.** DV3(b) smoothness is now measured quantitatively from frame-level position logging, and DV1(b) latency from round-trip percentiles. The one user-facing property still assessed only observationally is visible lag (the offset between the physical and displayed chair during motion); a high-speed video comparison at pilot speed would turn this into a measured frame-offset and close the last qualitative gap around DV1(b).
- **Drift-free swivel and yaw reference.** The spindle seat-tilt is now validated against an external spirit level, but the swivel angle and the compass-based bend correction are still checked against the drifting tablet gyroscope. A drift-free reference (digital protractor or laser angle meter) would give the swivel and the yaw correction the same external grounding the spindle already has.
- **Filter tuning grounded in measured inter-sample gaps and velocity profile.** Re-derive σ_a and the dead-reckoning decay constant from a measured distribution of inter-sample gaps and typical chair velocities (data for both is already present in the logs of the current validation runs), so the tuning is motivated rather than eyeballed.
- **Robustness testing.** Apply fault injection (corrupted bytes, forced disconnects, latency spikes) to verify the byte-level corruption handling and transport-layer recovery claims of Section 4.1 under adversarial conditions.
- **Historical playback.** Record telemetry sessions for later replay to enable post-incident analysis without a live connection.
- **Organisational embedding.** Adoption in practice requires integrating the live view into the service workflow: deciding where it runs (a service tablet at the lift or a remote support desk), how it is paired with a specific installation, and how its spatial output is combined with, rather than replacing, the existing text-and-code fault channel. A small pilot with a group of service engineers would surface these integration requirements before a wider rollout.

Step up to the functional problem. The recommendations above tighten the technical contribution; a final step returns to the functional problem that motivated it (Section 1): the difficulty service engineers face in interpreting text-based fault codes spatially. The validated pipeline is a necessary enabler for that goal (it places the chair on the rail to sub-millimetre pipeline accuracy, within 10 mm end-to-end, and renders fault state spatially), but whether it actually shortens diagnosis time or reduces on-site visits is an empirical human-factors question this report deliberately scoped out (Section 1). The natural follow-up is a small field study comparing diagnosis time with and without the live visualization on real service calls; the pipeline delivered here is the prerequisite that makes such a study possible.

Transfer to other domains. The technical core (mapping a single incremental-encoder count onto a non-uniform curved path with segment-dependent pulse costs, then interpolating and orienting it for a real-time display over a high-latency link) is not specific to stairlifts. The same problem shape recurs in elevators, platform lifts, gantry cranes, overhead conveyors, and automated guided ve-

hicles on curved tracks, all of which pair a curved travel path with encoder-based position feedback. The piecewise calibration map (Section 4.2) and the temporal Kalman/dead-reckoning interpolation (Section 4.3) are directly reusable there; only the segment-type pulse-cost table and the orientation axes would need to be reparameterised.

6 Discussion

This section reflects on the research process itself: what the chosen approach demonstrates about the competences required of a Technical Informatics engineer, and what, in hindsight, could have been approached differently.

6.1 Competences Demonstrated

The research set out from a functional problem (unreadable text-based fault output) and isolated a technical core (non-linear encoder mapping, recursive state estimation, singularity-free orientation reconstruction). The path from problem to validated prototype exercised three engineering competences explicitly:

- *Formal analysis grounded in literature.* Each of the three mathematical models (Kalman, Catmull-Rom, quaternions) was selected against alternatives through a comparison of established properties, not personal preference. The selections are traceable to cited sources, and the derivations in Section 4.3 are reproducible.
- *Architectural decomposition.* The five-layer pipeline isolates concerns by design: each identified failure mode (bit-level corruption, message-level corruption, connection drop, sample-rate / inter-sample gap, frame drop) maps to a single layer, which is what makes layer-local containment possible in principle. Demonstration of containment under adversarial conditions is a separate task that this report does not undertake.
- *Validation against measurable thresholds.* The pass/fail criteria are quantitative and, for the decisive second campaign, pre-registered before the runs rather than calibrated to the results. Where the original criteria proved ill-suited to the sensor they were reformulated and re-registered transparently (Section 5, Limitations) rather than quietly adjusted. Measured values are reported in Section 4.4 with explicit pass/fail labels against the criteria as defined.

6.2 What Could Have Been Done Differently

The limitations listed in Section 5 describe what the present runs do and do not establish; this subsection reflects on the process decisions behind them, not the limitations themselves.

The central process lesson is that two of the three empirical instrument criteria (DV1(b), DV3(b)) were specified before the sensor and pipeline behaviour were sufficiently understood. The original 100 ms Bluetooth-round-trip threshold and the tenfold Kalman jitter-reduction threshold both encoded implicit assumptions (a last-value-wins display, an analog noisy sensor) that did not hold for the system that was actually built. The reformulated v2 cri-

teria (percentile latency and a velocity-based smoothness test) are precisely the quantitative forms a careful upfront analysis would have produced, but they were arrived at only after the first campaign exposed the mismatch, and pre-registered one campaign later than ideal. A brief characterisation of the inter-sample gap distribution and the chair-velocity profile at the outset would have caught both mismatches before any run, and would also have given a measured rather than eyeballed basis for the filter tuning parameters (σ_n , dead-reckoning decay constant, the 0.85/0.15 spindle blend).

A secondary lesson concerns coverage: the first campaign's two runs on a single rail were a correctness-of-concept choice, not a generalisability one. The second campaign added a longer rail with a different bend composition, which is why the report can now report results on two geometries, but reserving test-bench time across several rails from the start would have been the more defensive plan. Both lessons point the same way: budget upfront time for sensor and rail characterisation before locking in validation criteria.

6.3 Scope Boundaries

The research deliberately excluded the shipping application itself (the *beroepsproduct*, or professional deliverable in the HBO-TI sense) from the scope of the research questions. Source code is referenced where it implements a research instrument, but the application as a whole is a professional deliverable for the stakeholder, not a research product (*onderzoeksproduct*). This separation was maintained throughout the article in line with the format guidelines [28] and prevents the research from drifting into an application showcase.

Bibliography

- [1] B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, 1988.
- [2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems* (F.-J. Kahlen, S. Flumerfelt, and A. Alves, eds.), pp. 85–113, Cham: Springer, 2017.
- [3] J. Bofill, M. Abisado, J. Villaverde, and G. A. Sampe-dro, "Exploring digital twin-based fault monitoring: Challenges and opportunities," *Sensors*, vol. 23, no. 16, p. 7087, 2023.
- [4] W. Elmenreich, "An introduction to sensor fusion," Research Report 47/2001, Vienna University of Technology, Institut für Technische Informatik, 2002.
- [5] R. J. E. Merry, M. J. G. van de Molengraft, and M. Steinbuch, "Velocity and acceleration estimation for optical incremental encoders," *Mechatronics*, vol. 20, no. 1, pp. 20–26, 2010.
- [6] W. Bolton, *Mechatronics: Electronic Control Systems in Mechanical and Electrical Engineering*. Pearson Education, 7th ed., 2019.
- [7] Bluetooth SIG, "Serial port profile," Adopted Specification v1.2, Bluetooth Special Interest Group, 2022.
- [8] G. Strang and E. Herman, *Calculus Volume 2*. OpenStax, 2016. Section 3.6: Numerical Integration. Open access, CC BY-NC-SA 4.0.
- [9] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA: California Technical Publishing, 1997. Full text available free online.
- [10] P. Narkhede, S. Poddar, R. Walambe, G. Ghinea, and K. Kotecha, "Cascaded complementary filter architecture for sensor fusion in attitude estimation," *arXiv preprint arXiv:2107.03970*, 2021. Open access, CC BY 4.0.
- [11] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Transactions of the ASME – Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [12] G. Welch and G. Bishop, "An introduction to the Kalman filter," Tech. Rep. TR 95-041, University of North Carolina at Chapel Hill, 1995. Updated 2006.
- [13] L. Piegl, "On NURBS: A survey," *IEEE Computer Graphics and Applications*, vol. 11, no. 1, pp. 55–71, 1991.
- [14] E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design* (R. E. Barnhill and R. F. Riesenfeld, eds.), pp. 317–326, New York: Academic Press, 1974.
- [15] J. Diebel, "Representing attitude: Euler angles, unit quaternions, and rotation vectors," tech. rep., Stanford University, October 2006.

- [16] K. Shoemake, “Animating rotation with quaternion curves,” in *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, pp. 245–254, ACM, 1985.
- [17] J. B. Kuipers, *Quaternions and Rotation Sequences: A Primer with Applications to Orbits, Aerospace, and Virtual Reality*. Princeton University Press, 1999.
- [18] Khronos Group, “glTF 2.0 specification,” tech. rep., Khronos Group, 2021. .glb binary container used for the DeVi Comfort chair model.
- [19] J. Linietsky and A. Manzur, “Godot engine: Multiplatform 2d and 3d open source game engine.” Software, MIT License, 2014. Version 4.6 used in this work. Licence text at <https://godotengine.org/license/>; feature flag "4.6", "Forward Plus" in `project.godot`.
- [20] Unity Technologies, “Unity engine 6 – licensing and platform support.” Vendor documentation, 2024. Seat-fee model and revenue-threshold conditions.
- [21] Epic Games, “Unreal engine 5 – end user license agreement.” Vendor documentation, 2022. 5% royalty over USD 1M per product per year.
- [22] R. Cabello, “three.js – JavaScript 3d library.” Software, MIT License, 2010. Browser/WebGL target only; native mobile requires WebView wrapper.
- [23] J. Schreier, “Unity reverses course on controversial game-engine fees.” Bloomberg, 2023. Context on 2023 Unity Runtime Fee reversal; cited as licence-stability risk factor.
- [24] R. D. Graham and P. C. Johnson, “Finite state machine parsing for internet protocols: Faster than you think,” in *IEEE Symposium on Security and Privacy Workshops*, pp. 185–190, IEEE, 2014.
- [25] R. R. Labbe, *Kalman and Bayesian Filters in Python*. Self-published (GitHub), 2015. Chapter 7: Continuous White Noise Model. CC BY 4.0 license.
- [26] Y. Bar-Shalom, X.-R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. New York: John Wiley & Sons, 2001.
- [27] National Imagery and Mapping Agency, *The American Practical Navigator*. Bethesda, MD: National Imagery and Mapping Agency, bicentennial ed., 2002. Chapter 7: Dead Reckoning. U.S. Government, public domain.
- [28] A. M. Gieling, “Onderzoeksartikelformat (t.b.v. tingenieurs) v.2223.” Inholland University of Applied Sciences, Technical Informatics, 2023. Course-provided format specification for the HBO-TI research article.